

## Problem A. Greatest Common Divisor

Input file:            standard input  
Output file:           standard output  
Time limit:            1 second  
Memory limit:         256 megabytes

Nurlan is helping his younger brother with a homework problem: given two positive integers  $a$  and  $b$ , find their **greatest common divisor** — the largest positive integer that divides both of them.

His brother wrote a straightforward program: check every integer from  $\min(a, b)$  down to 1 and stop at the first common divisor. For  $\text{gcd}(12, 18)$  it works fine — at most 12 checks. For  $\text{gcd}(1000000007, 1000000009)$  it already has to check about a billion numbers, and for the values in this problem it would never finish at all.

Help Nurlan and write a program that is fast enough.

### Input

The only line contains two integers  $a$  and  $b$  ( $1 \leq a, b \leq 10^{18}$ ).

### Output

Print a single integer — the greatest common divisor of  $a$  and  $b$ .

### Examples

| standard input        | standard output |
|-----------------------|-----------------|
| 1000000007 1000000009 | 1               |
| 12 18                 | 6               |

### Note

In the first example both numbers are prime, so their only common divisor is 1.

In the second example  $12 = 2 \cdot 2 \cdot 3$  and  $18 = 2 \cdot 3 \cdot 3$ , so the largest common divisor is  $2 \cdot 3 = 6$ .

## Problem B. Binary Exponentiation

Input file:            **standard input**  
Output file:           **standard output**  
Time limit:            1 second  
Memory limit:         256 megabytes

Compute  $a^n \bmod m$ .

The straightforward way — multiply  $a$  by itself  $n$  times — needs  $O(n)$  operations. For  $n$  up to  $10^{18}$  that is hopeless: even at a billion multiplications per second it would take more than thirty years.

Binary exponentiation does the same job in  $O(\log n)$  multiplications. It is based on a simple observation: if  $n$  is even, then  $a^n = (a^{n/2})^2$ , and if  $n$  is odd, then  $a^n = a \cdot a^{n-1}$ . For example,  $2^{10} = (2^5)^2 = (2 \cdot 2^4)^2 = (2 \cdot (2^2)^2)^2$  — only four multiplications instead of ten.

### Input

The only line contains three integers  $a$ ,  $n$  and  $m$  ( $1 \leq a \leq 10^{18}$ ,  $0 \leq n \leq 10^{18}$ ,  $1 \leq m \leq 10^9$ ).

### Output

Print a single integer — the value of  $a^n \bmod m$ .

### Examples

| standard input  | standard output |
|-----------------|-----------------|
| 2 10 1000000000 | 1024            |
| 5 0 1           | 0               |

### Note

In the first example  $2^{10} = 1024$ , and  $1024 \bmod 10^9 = 1024$ .

In the second example  $m = 1$ . Every integer is divisible by 1, so the remainder is 0 — this holds for  $a^0 = 1$  as well.

## Problem C. Prime or Not

Input file:            **standard input**  
Output file:           **standard output**  
Time limit:            1 second  
Memory limit:         256 megabytes

Asman is very curious and likes to check whether a number is prime or not.

Recall that a **prime number** is an integer greater than 1 that has exactly two positive divisors: 1 and the number itself. In particular, 1 is not prime, because it has only one positive divisor.

Given an integer  $a$ , determine whether it is prime.

### Input

The only line of the input contains one integer  $a$  ( $1 \leq a \leq 10^6$ ).

### Output

Print “YES” (without quotes) if  $a$  is prime, and “NO” (without quotes) otherwise.

### Examples

| standard input | standard output |
|----------------|-----------------|
| 1              | NO              |
| 10             | NO              |
| 2              | YES             |
| 97             | YES             |

### Note

In the first example  $a = 1$ . It has only one positive divisor, so it is not prime.

In the second example  $10 = 2 \cdot 5$  is composite.

In the third and fourth examples 2 and 97 are prime.

## Problem D. Prime Numbers

Input file:            **standard input**  
Output file:           **standard output**  
Time limit:            1 second  
Memory limit:         256 megabytes

Recall that a **prime number** is an integer greater than 1 that has exactly two positive divisors: 1 and the number itself. Written in increasing order, the prime numbers form the sequence 2, 3, 5, 7, 11, 13, 17, ...

Given an integer  $n$ , print the  $n$ -th number of this sequence.

### Input

The only line contains a single integer  $n$  ( $1 \leq n \leq 1000$ ).

### Output

Print a single integer — the  $n$ -th prime number.

### Example

| standard input | standard output |
|----------------|-----------------|
| 5              | 11              |

### Note

The prime numbers in increasing order are 2, 3, 5, 7, 11, 13, ..., so the 5-th of them is 11.

Do not confuse the index with the value: the 100-th prime number is 541 and the 1000-th is 7919. Every answer fits into a 32-bit integer.

## Problem E. Prime Factorization

Input file:            **standard input**  
Output file:           **standard output**  
Time limit:            1 second  
Memory limit:         256 megabytes

Every integer greater than 1 can be written as a product of prime numbers, and this representation is unique up to the order of the factors. For example,  $60 = 2 \cdot 2 \cdot 3 \cdot 5$  and  $210 = 2 \cdot 3 \cdot 5 \cdot 7$ .

Given an integer  $n$ , print its prime factorization.

Trying every divisor from 2 to  $n$  is far too slow for  $n$  up to  $10^{12}$ . A much smaller bound is enough — but be careful: once you stop, what is left of  $n$  may still be a prime factor that has to be printed.

### Input

The only line contains a single integer  $n$  ( $2 \leq n \leq 10^{12}$ ).

### Output

Print the prime factors of  $n$  in non-decreasing order, separated by single spaces. If a prime divides  $n$  several times, print it that many times.

### Examples

| standard input | standard output |
|----------------|-----------------|
| 60             | 2 2 3 5         |
| 210            | 2 3 5 7         |
| 999999999989   | 999999999989    |

### Note

In the first example  $60 = 2 \cdot 2 \cdot 3 \cdot 5$ , and in the second one  $210 = 2 \cdot 3 \cdot 5 \cdot 7$ .

In the third example  $n$  is a prime number, so its factorization consists of  $n$  itself.

## Problem F. Equal Strings

Input file:            **standard input**  
Output file:         **standard output**  
Time limit:          1 second  
Memory limit:       256 megabytes

You are given two strings typed on a keyboard. Each string consists of lowercase English letters and the # symbol, where # means that the backspace key was pressed: it deletes the last character typed so far. If backspace is pressed when nothing has been typed yet, nothing happens.

Determine whether the two strings produce the same text.

### Input

The first line contains the string  $s_1$  and the second line contains the string  $s_2$  ( $1 \leq |s_1|, |s_2| \leq 10^5$ ).

Both strings consist of lowercase English letters and the # symbol.

### Output

Print “Yes” if the two strings produce the same text, and “No” otherwise.

### Examples

| standard input | standard output |
|----------------|-----------------|
| abc##<br>a#b#a | Yes             |
| ab#c<br>ad#c   | Yes             |
| a#c<br>bb##    | No              |

### Note

In the first example **abc##** becomes **a**, and **a#b#a** also becomes **a**, so the answer is **Yes**.

In the second example **ab#c** becomes **ac** and **ad#c** becomes **ac**.

In the third example **a#c** becomes **c**, while **bb##** becomes the empty text, so the answer is **No**.

## Problem G. Balanced Sequence of Letters

Input file:            **standard input**  
Output file:           **standard output**  
Time limit:            1 second  
Memory limit:         256 megabytes

You are given a string  $S$  consisting of lowercase Latin letters. Determine whether it is *balanced*.

A string is balanced if and only if it can be built by the following rules:

- the empty string is balanced;
- if the strings  $s$  and  $t$  are balanced, then their concatenation  $st$  is also balanced;
- if the string  $s$  is balanced, then the string  $xsx$  is balanced for any lowercase Latin letter  $x$ .

For example, the string **abba** is balanced, but the string **abbb** is not.

### Input

The only line contains the string  $S$  ( $1 \leq |S| \leq 10^5$ ).

### Output

Print “YES” if the string is balanced, and “NO” otherwise.

### Examples

| standard input | standard output |
|----------------|-----------------|
| sbaabsss       | YES             |
| sbabasss       | NO              |
| baab           | YES             |
| abpa           | NO              |

### Note

In the first example **sbaabsss** is the concatenation of **sbaabs** and **ss**. Here **sbaabs** is built as  $xsx$  with  $x = \mathbf{s}$  around the balanced string **baab**, and **ss** is built as  $xsx$  with  $x = \mathbf{s}$  around the empty string.

In the third example **baab** is built as  $xsx$  with  $x = \mathbf{b}$  around the balanced string **aa**.

The strings **sbabasss** and **abpa** of the second and fourth examples cannot be built by these rules.

## Problem H. Nugman and Stack

Input file:            `standard input`  
Output file:          `standard output`  
Time limit:           2 seconds  
Memory limit:        256 megabytes

One day Nugman was solving the problems of LAB1 and managed to finish almost all of them. Only one problem was left, and it would not give in — so Nugman asks for your help.

There are  $N$  people standing in a queue, and the  $i$ -th of them is  $a_i$  years old. The queue starts at position 1. Every person wants to know the age of the closest person standing before them who is **strictly younger** — that is, the nearest position to the left holding a strictly smaller age. If there is no such person, the answer is  $-1$ .

### Input

The first line contains a single integer  $N$  ( $1 \leq N \leq 10^5$ ) — the number of people in the queue.

The second line contains  $N$  integers  $a_1, a_2, \dots, a_N$  ( $1 \leq a_i \leq 10^9$ ) — the ages of the people, listed from the front of the queue to the back.

### Output

Print  $N$  integers separated by spaces, where the  $i$ -th of them is the answer for the  $i$ -th person.

### Examples

| standard input | standard output |
|----------------|-----------------|
| 5<br>2 1 5 8 3 | -1 -1 1 5 1     |
| 5<br>1 2 3 4 5 | -1 1 2 3 4      |

### Note

In the first example the queue is  $[2, 1, 5, 8, 3]$ . The first two people have nobody strictly younger before them. For the third person (age 5) the nearest smaller age to the left is 1; for the fourth (age 8) it is 5; for the fifth (age 3) the ages 8 and 5 are too large, so the answer is 1 again.

In the second example the ages increase, so every person sees the one standing directly in front of them.

Equal ages do not count. For the queue  $[3, 1, 2, 2]$  the answer is  $[-1, -1, 1, 1]$ : for the fourth person the third one has the same age, not a smaller one.

## Problem I. Royal Flush

Input file:            **standard input**  
Output file:           **standard output**  
Time limit:            1 second  
Memory limit:          256 megabytes

You are in a casino and the croupier offers you a game. There is a deck of  $N$  cards numbered from 1 to  $N$ , and the croupier shows you a trick:

- take the top card and put it at the bottom of the deck;
- take the new top card — it turns out to be the card numbered 1 — and put it aside;
- take the top two cards and move them to the bottom of the deck, one by one;
- take the new top card — it turns out to be the card numbered 2 — and put it aside;
- ...
- repeat until no cards are left.

In other words, at step  $i$  the croupier moves  $i$  cards from the top to the bottom one by one and then puts the new top card aside, and that card must be the card numbered  $i$ .

Note that at step  $i$  the deck may contain fewer than  $i$  cards. In that case some cards are moved to the bottom several times, see the notes.

You are asked to repeat the trick. Find an initial arrangement of the deck that makes it work.

### Input

The first line of the input contains the number of test cases  $T$  ( $1 \leq T \leq 100$ ). Each of the next  $T$  lines contains a single integer  $N$  ( $1 \leq N \leq 1000$ ) — the size of the deck for that test case.

### Output

For each test case print  $N$  space-separated integers on a separate line — the order of the deck from the top card to the bottom one.

It can be shown that such an arrangement always exists, so you never have to report that the trick is impossible.

### Examples

| standard input | standard output |
|----------------|-----------------|
| 2              | 2 1 4 3         |
| 4              | 3 1 4 5 2       |
| 5              |                 |
| 3              | 3 1 4 5 2       |
| 5              | 4 1 6 3 2 5     |
| 6              | 4 1 6 3 2 5     |
| 6              |                 |

### Note

In the first test case of the example the deck  $[2, 1, 4, 3]$  is processed as follows:

- the initial deck is  $[2, 1, 4, 3]$ ;

- move one card to the bottom:  $[1, 4, 3, 2]$ ;
- put the card 1 aside:  $[4, 3, 2]$ ;
- move two cards to the bottom, one by one:  $[2, 4, 3]$ ;
- put the card 2 aside:  $[4, 3]$ ;
- move three cards to the bottom, one by one:  $[4, 3] \rightarrow [3, 4] \rightarrow [4, 3] \rightarrow [3, 4]$ ;
- put the card 3 aside:  $[4]$ ;
- move four cards to the bottom — the only remaining card just returns to its place — and put the card 4 aside: the deck is empty.

## Problem J. Boris vs Nursik

Input file:            **standard input**  
Output file:         **standard output**  
Time limit:          1 second  
Memory limit:       256 megabytes

Boris and Nursik are playing the card game “drunkard”. All the cards are divided equally between the two players. On every move both players reveal the top card of their deck, and the one whose card is higher takes both revealed cards and puts them at the bottom of their own deck. The player who is left without cards loses.

The winner of a move first puts Boris’s card at the bottom of the deck and then Nursik’s card, so Nursik’s card ends up lowest.

The game is played with 10 cards with values from 0 to 9. A larger card beats a smaller one, with a single exception: the card 0 beats the card 9.

Write a program that simulates the game and determines the winner.

### Input

The first line contains 5 integers separated by spaces — Boris’s cards. The second line contains Nursik’s 5 cards in the same format.

Every card value is an integer between 0 and 9, and all 10 values are distinct. The cards are listed from top to bottom, that is, each line starts with the card that will be revealed first.

### Output

Print the word **Boris** or **Nursik** — the name of the winner — followed by the number of moves made before the win.

It can be shown that with 10 distinct cards the game always ends: no deal leads to an infinite loop.

### Example

| standard input         | standard output |
|------------------------|-----------------|
| 1 3 5 7 9<br>2 4 6 8 0 | Nursik 5        |

### Note

In the example Boris holds [1, 3, 5, 7, 9] and Nursik holds [2, 4, 6, 8, 0]. Nursik wins the first four moves ( $2 > 1$ ,  $4 > 3$ ,  $6 > 5$ ,  $8 > 7$ ), and on the fifth move Boris’s last card 9 loses to 0 by the special rule. Boris is left without cards, so Nursik wins after 5 moves.